SOFTWARE ENGINEERING TESTING DELIVERABLE

Electronic Store Inventory Management System

System Validation Report

Project Title	Electronic Store Inventory Management System	**Document Title**	System Validation Report
Testing Date	Final Academic Submission	**Document ID**	SVR-ESIMS-01
Specification Basis	SRS Table (24 Requirements: FR-001–FR-012, NFR-001–NFR-012) & Security SRS (SEC-001–SEC-016)	**Test Plan Basis**	18-Case Test Case Document (`Documents/test_plan.pdf`: 2 UT, 12 IT, 4 ST)
Automated Evidence	Pytest 9.1.1 Suite: 373 Passed / 0 Failed / 0 Errors / 0 Skipped (120.60s)	**SRS Validation Summary**	**13 PASS / 7 PARTIAL / 0 FAIL / 4 NOT TESTED (Total: 24)**

1. Introduction

This System Validation Report evaluates whether the implemented **Electronic Store Inventory Management System** satisfies the functional and non-functional requirements specified in the project's Software Requirements Specification (`Documents/srs_table.pdf`) and supporting architecture and security specifications (`Documents/security_requirements_specifications.pdf`, `Documents/se_electronic_store_arch_justification.pdf`, `Documents/Use_Case_Flow.pdf`, `Documents/use_case_diagram.pdf`, and `Documents/electronic_store_component_diagram.pdf`).

Whereas the companion *Unit Test Report* (`docs/unit_test_report.pdf`) focuses on module-level automated test execution across the 20 Pytest test modules (373 automated tests), this report maps both the 18 formal test cases in `Documents/test_plan.pdf` (`UT_01`–`UT_02`, `IT_01`–`IT_12`, and `ST_01`–`ST_04`) and the 373 automated Pytest tests directly against all 24 SRS requirements (`FR-001` through `FR-012` and `NFR-001` through `NFR-012`). Strict evidence rules are applied: a requirement is marked **PASS** only when its acceptance criteria are verified by actual test evidence, **PARTIAL** when only a subset of acceptance criteria is verified, and **NOT TESTED** when execution evidence is absent.

2. System Validation Objectives

The specific objectives of this validation assessment are to:

- **Verify Implemented Functional Requirements:** Confirm that all 12 functional requirements (`FR-001`–`FR-012`) operate correctly across domain services, HTTP routes, and database persistence.
- **Validate System Behaviour Against SRS Acceptance Criteria:** Evaluate each requirement against the exact acceptance criteria defined in `Documents/srs_table.pdf` rather than relying solely on source code presence.
- **Verify Interactions Between Major System Modules:** Confirm that cross-module workflows—such as billing with stock deduction and low-stock alert generation, purchase order receipt with stock replenishment, and product edits with audit logging—maintain data consistency.
- **Validate Authentication and Role-Based Access Control (RBAC):** Verify credential verification, deactivated-user blocking, password hashing, and server-side permission boundaries across `admin`, `manager`, and `staff` roles.
- **Verify Inventory Consistency, Sales, Purchase Orders, and Reports:** Confirm transactional atomicity on failed multi-item sales and restocks, accurate refund and stock updates on returns, and exact date-range aggregations across Sales, Stock, and Revenue reports.
- **Identify Partially Verified and Untested Requirements:** Explicitly document non-functional and security criteria (such as 99% uptime, 50-user concurrency, 15-minute staff onboarding usability, full backup restoration, and HTTPS/TLS encryption in transit) that remain partially verified or untested.

3. System Under Test

Per `Documents/se_electronic_store_arch_justification.pdf` and `Documents/electronic_store_component_diagram.pdf`, the Electronic Store Inventory Management System is implemented in Python/Flask using a **Three-Tier Layered Architecture**: a **Presentation Layer** (`app/presentation/routes/`, `app/presentation/authz.py`, and Jinja2 templates in `app/templates/`), a **Business Logic Layer** (`app/business/`), and a **Data Access Layer** (`app/data/models/` and `app/data/extensions.py`). The existence of an implementation module does not by itself establish validation; each module's validation status is determined strictly by test evidence in Sections 5 and 6.

- **Authentication and User Management (**`app/presentation/routes/auth.py`**,** `app/presentation/authz.py`**,** `app/business/security.py`**,** `app/data/models/user.py`**):** Provides session login/logout (`/login`, `/logout`), salted `bcrypt` password hashing, Admin-only user creation/role editing/deactivation (`/users*`), token-based password reset (`/forgot-password`, `/reset-password/<token>`), and `@admin_required` / `@manager_required` decorators.
- **Product and Inventory Management (**`app/presentation/routes/main.py`**,** `app/data/models/product.py`**):** Supports product creation, editing, deletion (blocked if sales history exists), SKU uniqueness validation, search by name/SKU (`?q=`), filtering by category and stock status (`in_stock`, `low_stock`, `out_of_stock`), reorder threshold configuration, and dashboard low-stock alert banners.
- **Sales and Billing (**`app/business/sales.py`**,** `app/data/models/sale.py`**,** `app/presentation/routes/main.py`**):** Processes single- and multi-item sales (`create_sale()`, `POST /sales`), validates positive integer quantities and stock availability, computes subtotal, 5% tax, and grand total, deducts stock atomically, and generates itemized invoices (`GET /sales/<id>/invoice`).
- **Purchase Orders (**`app/business/purchase_order.py`**,** `app/data/models/purchase_order.py`**,** `app/presentation/routes/purchase_order.py`**):** Enables Managers/Admins to create purchase orders (`PO-XXXX`) with `Pending` status (manually or directly from low-stock alerts), record full receipts (`receive_purchase_order()`) or partial deliveries (`record_partial_delivery()`), and increment product stock upon receipt.
- **Supplier Management (**`app/presentation/routes/supplier.py`**,** `app/data/models/supplier.py`**):** Provides Admin-only routes (`/suppliers*`) for adding, viewing, and editing suppliers, validating field lengths, and linking products to suppliers via `Product.supplier_id`.
- **Returns and Refunds (**`app/business/sales.py:process_return()`**,** `app/data/models/product_return.py`**,** `POST /returns`**):** Processes sale-linked and standalone product returns, validates remaining returnable quantities, blocks duplicate `return_id` submissions, restores stock, and records `refund_amount` based on the original sale unit price.
- **Reports (**`app/presentation/routes/main.py`**,** `app/business/export_service.py`**,** `app/presentation/routes/purchase_order.py`**):** Generates Manager/Admin date-range reports for Sales (`/reports/sales`), Stock Movements (`/reports/stock`), and Revenue (`/reports/revenue`), along with CSV and PDF report exports (`/reports/export/<type>/<fmt>`).
- **Audit Logging (**`app/business/audit.py`**,** `app/data/models/audit_log.py`**):** Records immutable `AuditLog` database entries capturing authenticated actor `user_id`, `product_id`, `action` (`stock_edit`, `price_change`, `product_delete`, `stock_restock`), `details`, and UTC `timestamp`.
- **Database Backup (**`scripts/backup_database.py`**):** Standalone operational utility (`create_backup()`) that copies the SQLite database file to `backups/inventory_backup_YYYYMMDD_HHMMSS.db` using `sqlite3.Connection.backup()`.

4. Validation Environment

Validation evidence in this report originates from two distinct environments that must be clearly distinguished: (1) the **Automated Test Environment** executed directly in the repository's virtual environment via Pytest (`docs/unit_test_report.pdf`), which uses Flask's in-process WSGI `test_client()` and isolated SQLite databases without a live network socket or browser engine; and (2) the **User-Reported Manual Verification Environment**, in which the user manually executed the 18 test cases in `Documents/test_plan.pdf` against the running application (and documented subset browser checks in `README.md`). Verification is established through the automated test runner and documented verification logs. Table 1 summarizes both environments.

Environment Parameter	**Automated Test Environment (Executed by Pytest)**	**User-Reported Manual Environment**
Operating System	macOS and Windows (both were used)	Local desktop OS hosting Flask dev server (`http://127.0.0.1:5000/`)

Environment Parameter	Automated Test Environment (Executed by Pytest)	User-Reported Manual Environment
Python & Virtual Env	Python 3.14.6 in repository local `./venv` (`./venv/bin/python`)	Python 3.12+ compatible local virtual environment (`python run.py`)
Web & Auth Stack	`Flask 3.1.3`, `Werkzeug 3.1.9`, `Flask-Login 0.6.3`, `Flask-WTF 1.3.0`, `WTForms 3.3.1`, `Jinja2 3.1.6`	Same Flask 3.1.3 application stack served over local HTTP
Database Engine	SQLite 3 (`sqlite:///:memory:` per test in `tests/conftest.py`; `tmp_path` SQLite file in `test_backup_database.py`) via `SQLAlchemy 2.1.6`	Local SQLite database (`instance/inventory.db`) / SQLAlchemy ORM
Test Runner / Client	`pytest 9.1.1` (`./venv/bin/python -m pytest -v`) using Flask `app.test_client()`; 373 passed in 120.60s	User-reported manual execution of 18 test cases; `README.md` notes Chrome 156.0.8078.79, Firefox 159.0, Edge 156.0.4314.50 for 5 UI flows
Security & Utilities	`bcrypt 5.0.0`, `PyMySQL 1.2.3`, `python-dotenv 1.2.4`, `reportlab 5.1.0`, `unittest.mock` (SMTP mock)	Same application dependencies; no production HTTPS/TLS proxy or load generator

5. Test Case Validation Summary (18-Case Test Plan)

Table 2 presents all 18 test cases from the latest Test Case Document (`Documents/test_plan.pdf`), preserving the exact Test Case IDs (`UT_01`–`UT_02`, `IT_01`–`IT_12`, `ST_01`–`ST_04`), testing levels, module names, test descriptions, and expected results. In the *Actual Result* column, user-reported manual verification results are explicitly distinguished from automated Pytest execution evidence.

Test Case ID	Testing Level	Module	Test Description	Expected Result	Actual Result (Manual & Automated Evidence)	Status
`UT_01`	Unit Testing	Stock Management	To test stock deduction in the sales service after a sale	Sale record is returned and product stock updates from 10 to 6	**Document / User-Reported Manual:** `create_sale()` completed the sale and product stock decreased from 10 to 6. **Automated Pytest:** Verified in `tests/test_sales.py` by `test_stock_auto_reduced_after_single_item_sale` (PASS).	**PASS**
`UT_02`	Unit Testing	Sales Validation	Reject sale exceeding available stock	SaleValidationError ("Insufficient stock...") is raised; sale is rejected and stock remains 4	**Document / User-Reported Manual:** Overselling was rejected with `SaleValidationError`; no sale was created and stock remained 4. **Automated Pytest:** Verified in `tests/test_sales.py` by `test_sale_rejected_when_insufficient_stock` (PASS).	**PASS**
`IT_01`	Integration Testing	User Authentication	To test login functionality with valid credentials	User is authenticated, session is established, and user is redirected to dashboard (/)	**User-Reported Manual:** Redirected to dashboard (`/`). **Automated Pytest:** `tests/test_auth.py` (`test_correct_password_logs_in` and `conftest.py:logged_in_staff`) verified `POST /login` returned HTTP 200 and established session `user_id` (dashboard content not asserted by `test_correct_password_logs_in`) (PASS).	**PASS**
`IT_02`	Integration Testing	User Authentication	To test login with invalid password	Error message: "Invalid username or password, or account deactivated."	**User-Reported Manual:** Login was rejected and "Invalid username or password, or account deactivated." was displayed. **Automated Pytest:** Verified in `tests/test_auth.py` by `test_wrong_password_is_rejected` (`deshna-test` / `WrongPassword123`) (PASS).	**PASS**

Test Case ID	Testing Level	Module	Test Description	Expected Result	Actual Result (Manual & Automated Evidence)	Status
IT_03	Integration Testing	Product Management	To test adding a new product with valid data	Product is saved in DB and flash message "Product created successfully." is displayed	**User-Reported Manual:** Product was saved in DB with SKU `SPK-BT-100`, price 39.99, stock 20, and confirmation message was displayed. **Automated Pytest:** Verified in `tests/test_products.py` by `test_create_product_with_sku_success` (PASS).	**PASS**
IT_04	Integration Testing	Product Management	To test adding a product with duplicate SKU	Error message: "A product with this SKU already exists." and duplicate is not saved	**User-Reported Manual:** Duplicate SKU was rejected with "A product with this SKU already exists."; no duplicate product was created. **Automated Pytest:** Verified in `tests/test_products.py` by `test_create_product_rejects_duplicate_sku` (PASS).	**PASS**
IT_05	Integration Testing	Input Validation	Verify clear validation for invalid product or sale input	Clear validation message is displayed ("Product name is required." / positive quantity); invalid record is not saved	**User-Reported Manual:** Invalid input was rejected with a validation message and no invalid product or sale record was saved. **Automated Pytest:** Verified by `test_create_product_rejects_empty_name` (`test_products.py`) and `test_sale_route_rejects_invalid_quantity` (`test_sales.py`) (PASS).	**PASS**
IT_06	Integration Testing	Audit Logging	Verify product or stock changes create an audit entry	AuditLog DB record stores action="stock_edit", actor user_id, timestamp, and details "5 to 10"	**User-Reported Manual:** Stock update created an `AuditLog` record in DB with the authenticated actor `user_id` and "Stock changed from 5 to 10". **Automated Pytest:** Verified in `tests/test_products.py` by `test_stock_edit_creates_audit_log_entry` (PASS).	**PASS**
IT_07	Integration Testing	Billing + Inventory Integration	To test invoice generation and stock synchronization together	Invoice is generated ("Sales Invoice", subtotal 51.00) and stock is reduced by 2 (from 5 to 3) simultaneously	**User-Reported Manual:** Invoice was generated and displayed, and inventory stock in DB was reduced from 5 to 3. **Automated Pytest:** Verified in `tests/test_sales.py` by `test_create_sale_route_redirects_to_invoice_and_renders_details` (PASS).	**PASS**
IT_08	Integration Testing	Purchase Order + Inventory Integration	To test stock update when a Purchase Order is marked as Received	PO status becomes "Received" and product stock increases by 20 (from 14 to 34)	**User-Reported Manual:** Purchase Order `PO-0001` receipt completed and the received quantity (20) increased stock from 14 to 34. **Automated Pytest:** Verified in `tests/test_purchase_orders.py` by `test_receive_po_route_increases_stock` (PASS).	**PASS**
IT_09	Integration Testing	Authentication + Role Access Integration	To test restricted access enforcement across modules	Access denied with HTTP 403 Forbidden response.	**User-Reported Manual:** Staff access to the restricted Supplier module (`/suppliers`) was denied (HTTP 403). **Automated Pytest:** Verified in `tests/test_suppliers.py` by `test_suppliers_list_rejects_staff` (PASS).	**PASS**
IT_10	Integration Testing	Sales Transaction	Verify rollback of a failed multi-item sale	No partial update; stock remains unchanged (5 and 0) and no incomplete sale is committed	**User-Reported Manual:** Failed multi-item sale was rolled back; stock and sale records were not partially updated. **Automated Pytest:** Verified in `tests/test_sales.py` by `test_multi_item_sale_rolls_back_when_second_item_fails` and `test_sale_route_preserves_stock_on_failed_checkout` (PASS).	**PASS**

Test Case ID	Testing Level	Module	Test Description	Expected Result	Actual Result (Manual & Automated Evidence)	Status
IT_11	Integration Testing	Returns / Refunds	Verify return processing updates stock and records refund	Stock increases by 1 (from 7 to 8) and ProductReturn records quantity = 1 and refund_amount = 25.00	**User-Reported Manual:** Valid return was processed; stock increased to 8 and `ProductReturn` recorded `refund_amount` 25.00. **Automated Pytest:** Verified in `tests/test_sales.py` by `test_return_records_refund_from_original_sale_price` (PASS).	**PASS**
IT_12	Integration Testing	Database Backup	Verify successful database backup generation	Backup file is created, contains the source record ("test record",), and passes SQLite integrity check ("ok")	**User-Reported Manual:** Backup file was created; the known record was present and SQLite integrity check returned ok. **Automated Pytest:** Verified in `tests/test_backup_database.py` by `test_create_backup_successfully` (PASS).	**PASS**
ST_01	System Testing	Low Stock Alert System	To test alert generation when stock crosses the threshold	Low-stock alert banner ("Low-Stock Alert") is displayed on the dashboard for "Wireless Earphones"	**User-Reported Manual:** Low-stock alert was displayed on the dashboard when stock fell from 10 to 7 (below threshold 8). **Automated Pytest:** Verified in `tests/test_products.py` by `test_low_stock_alert_triggered_after_sale_reduces_stock_below_threshold` (PASS).	**PASS**
ST_02	System Testing	Reporting Module	To test sales report generation for a selected date range	Sales report is generated and displays matching transactions ("USB Keyboard") and calculated total (60.00) for the selected range	**User-Reported Manual:** Sales report displayed the matching sale details ("USB Keyboard") and calculated total (60.00). **Automated Pytest:** Verified in `tests/test_sales_report.py` by `test_sales_report_shows_sale_details_and_total` (PASS).	**PASS**
ST_03	System Testing	Role Access	Verify Staff, Manager and Admin permissions	Each role accesses permitted modules (HTTP 200) and is denied restricted actions (HTTP 403)	**User-Reported Manual:** Role-based access checks passed; restricted actions were denied (HTTP 403) for unauthorized roles. **Automated Pytest:** Verified across `test_auth.py`, `test_suppliers.py`, `test_purchase_orders.py`, `test_products.py`, and report test modules (PASS).	**PASS**
ST_04	System Testing	Stock Reporting	Verify stock movement report calculations	Report shows Units Sold (2), Restocked (5), Returned (1), Net Movement (4), and Current Stock (16)	**User-Reported Manual:** Stock report displayed the movement categories (2, 5, 1), calculated net movement (4), and current stock (16) correctly. **Automated Pytest:** Verified in `tests/test_stock_report.py` by `test_report_calculates_stock_movements` (PASS).	**PASS**

6. SRS Requirement Traceability Matrix

Table 3 maps all 24 Software Requirements Specification (SRS) requirements (`FR-001`–`FR-012` and `NFR-001`–`NFR-012` from `Documents/srs_table.pdf`) to the 18 formal test cases and the 373 automated Pytest tests. Statuses are assigned strictly according to verified acceptance criteria: **PASS** (acceptance criteria verified by execution evidence), **PARTIAL** (some aspects verified, but full acceptance criteria not proven), **FAIL** (evidence shows requirement violated), and **NOT TESTED** (no execution evidence available).

SRS Req ID	Requirement Description & Acceptance Criteria	Related Test Case ID(s)	Verification Evidence	Status	Remarks
FR-001	System shall allow authorized users to add, update, and delete product records (name, SKU, category, price, quantity, supplier). [Criteria: Product CRUD succeeds with valid input; duplicate SKU rejected.]	IT_03, IT_04, IT_05	User-reported manual pass (IT_03–IT_05). Automated Pytest in tests/test_products.py: test_create_product_success, test_create_product_with_sku_success, test_create_product_with_category_success, test_create_product_with_supplier_link, test_update_product_success, test_delete_product_success, test_create_product_rejects_duplicate_sku.	**PASS**	Product creation, editing, deletion (including sales-history protection), and duplicate SKU rejection are fully verified.
FR-002	System shall automatically update stock quantity on sale, return, or restock. [Criteria: Stock count reflects correct value within 2 sec of transaction.]	UT_01, IT_07, IT_08, IT_11	User-reported manual pass (UT_01, IT_07, IT_08, IT_11). Automated Pytest in tests/test_sales.py (test_stock_auto_reduced_after_single_item_sale, test_stock_auto_increased_after_return, test_stock_auto_increased_after_restocking) and tests/test_purchase_orders.py (test_receive_po_route_increases_stock).	**PASS**	Synchronous stock updates on sale, return, manual restock, and PO receipt are verified and execute in milliseconds locally (see NFR-001 for load SLA).
FR-003	System shall generate an alert when stock falls below a predefined reorder threshold. [Criteria: Alert triggers immediately when threshold condition met.]	ST_01	User-reported manual pass (ST_01). Automated Pytest in tests/test_products.py: test_is_below_reorder_threshold_boundaries, test_low_stock_alert_triggered_after_sale_reduces_stock_below_threshold, test_dashboard_shows_low_stock_alert_when_stock_below_threshold, test_low_stock_alert_clears_after_restock_above_threshold.	**PASS**	Immediate low-stock detection (stock < reorder_threshold) and dashboard alert rendering/clearing are verified.
FR-004	System shall allow staff to create a sale transaction and generate an invoice. [Criteria: Invoice generated with itemized list, total, date, transaction ID.]	UT_01, UT_02, IT_07, IT_10	User-reported manual pass (UT_01, UT_02, IT_07, IT_10). Automated Pytest in tests/test_sales.py: test_create_sale_route_redirects_to_invoice_and_renders_details, test_invoice_route_displays_all_required_fields_for_multi_item_sale.	**PASS**	Sale creation and invoice generation (INV-YYYYMMDD-XXXX, date, itemized list, subtotal, 5% tax, grand total) are verified.
FR-005	System shall authenticate users and restrict actions based on role (Admin, Manager, Staff). [Criteria: Unauthorized role access returns "Access Denied".]	IT_01, IT_02, IT_09, ST_03	User-reported manual pass (IT_01, IT_02, IT_09, ST_03). Automated Pytest in tests/test_auth.py, tests/test_suppliers.py, tests/test_purchase_orders.py, tests/test_products.py, and report test modules verifying login and HTTP 403 role enforcement.	**PASS**	Authentication and role-based HTTP 403 enforcement across Staff, Manager, and Admin roles are verified (see NFR-003 for endpoint coverage details).
FR-006	System shall allow admins to add/edit supplier details and link products to suppliers. [Criteria: Supplier record saved; linked products display correct supplier.]	IT_09 (RBAC only in 18-case PDF)	Automated Pytest in tests/test_suppliers.py (test_admin_can_create_supplier_with_all_fields, test_admin_can_edit_supplier_details, test_view_supplier_route) and tests/test_products.py (test_create_product_with_supplier_link, test_edit_product_supplier_link_and_unlink, test_supplier_displayed_in_inventory_table, test_supplier_detail_page_lists_linked_products).	**PASS**	Verified by automated tests in test_suppliers.py and test_products.py. The 18-case PDF covers only supplier RBAC (IT_09), not supplier CRUD/linking.

SRS Req ID	Requirement Description & Acceptance Criteria	Related Test Case ID(s)	Verification Evidence	Status	Remarks
FR-007	System shall allow managers to create purchase orders and mark them as received. [Criteria: PO created with "Pending" status; stock updates on "Received".]	IT_08	User-reported manual pass (`IT_08`). Automated Pytest in `tests/test_purchase_orders.py`: `test_create_purchase_order_creates_pending_po_without_updating_stock`, `test_create_po_route_creates_pending_po`, `test_receive_purchase_order_updates_status_and_increases_stock`, `test_receive_po_route_increases_stock`, `test_partial_delivery_sets_status_to_partially_received`.	**PASS**	PO creation with `Pending` status, full receipt transitioning to `Received` with stock update, and partial deliveries are verified.
FR-008	System shall allow users to search and filter products by name, SKU, category, or stock status. [Criteria: Search returns correct matching results within 1 sec.]	None in 18-case PDF (Automated Pytest)	Automated Pytest in `tests/test_products.py`: `test_search_by_exact_product_name`, `test_search_by_partial_product_name`, `test_search_by_exact_sku`, `test_search_by_partial_sku`, `test_search_case_insensitive`, `test_filter_by_category_exact`, `test_filter_by_stock_status_in_stock`, `test_filter_by_stock_status_low_stock`, `test_filter_by_stock_status_out_of_stock`, `test_filter_combined_with_search_query`.	**PASS**	Search (name/SKU) and filtering (category/stock status) are verified in `test_products.py` (not separately represented in the 18-case PDF; 10k-item `<1s` scale covered under `NFR-006`).
FR-009	System shall generate sales, stock, and revenue reports over a selectable date range. [Criteria: Report totals match underlying transaction records for selected range.]	ST_02, ST_04	User-reported manual pass (`ST_02`, `ST_04`). Automated Pytest in `tests/test_sales_report.py` (`test_sales_report_shows_sale_details_and_total`), `tests/test_stock_report.py` (`test_report_calculates_stock_movements`), and `tests/test_revenue_report.py` (`test_revenue_report_calculates_total_and_daily_breakdown`), plus CSV/PDF export tests in `test_purchase_orders.py`.	**PASS**	Sales, Stock, and Revenue report calculations and date-range filtering are verified (`ST_02` and `ST_04` cover Sales and Stock; `test_revenue_report.py` covers Revenue).
FR-010	System shall log key actions (stock edits, deletions, price changes) with timestamp and user ID. [Criteria: Every CRUD action creates a corresponding audit log entry.]	IT_06	User-reported manual pass (`IT_06`). Automated Pytest in `tests/test_products.py`: `test_stock_edit_creates_audit_log_entry`, `test_price_change_creates_audit_log_entry`, `test_product_deletion_creates_audit_log_entry`, `test_stock_and_price_edit_creates_both_audit_logs`, `test_audit_log_uses_authenticated_user_id_and_ignores_spoofed_form_user_id`.	**PASS**	All specified key actions (`stock_edit`, `price_change`, `product_delete`, plus `stock_restock`) record `user_id` and `timestamp` in `AuditLog`.
FR-011	System shall allow staff to process returns/refunds, updating stock and transaction records. [Criteria: Returned item quantity added back to stock; refund recorded.]	IT_11	User-reported manual pass (`IT_11`). Automated Pytest in `tests/test_sales.py`: `test_stock_auto_increased_after_return`, `test_return_route_updates_stock_and_blocks_duplicate`, `test_return_records_refund_from_original_sale_price`, `test_partial_returns_record_individual_refund_amounts`.	**PASS**	Return processing, stock increment, duplicate `return_id` prevention, and original-sale-price `refund_amount` persistence are verified.
FR-012	System shall allow password reset for users who are locked out of their accounts. [Criteria: User receives reset link/OTP and can set a new password successfully.]	None in 18-case PDF (Automated Pytest)	Automated Pytest in `tests/test_auth.py`: `test_forgot_password_sends_email` (mocks SMTP via `unittest.mock.patch`), `test_forgot_password_unknown_email_same_message`, `test_reset_password_changes_password`, `test_reset_password_rejects_mismatch`.	**PASS**	Signed token generation, reset link dispatch (mocked SMTP), and password update are verified in `test_auth.py` (not in 18-case PDF).

SRS Req ID	Requirement Description & Acceptance Criteria	Related Test Case ID(s)	Verification Evidence	Status	Remarks
NFR-001	System shall process a sale transaction and update inventory within 2 seconds. [Criteria: Transaction completion time <= 2 sec under normal load.]	UT_01, IT_07	Functional sale and stock updates complete in milliseconds in `tests/test_sales.py` and in the local single-user script `benchmarks/sales_performance.py` (1–100 items on SQLite). However, no automated test asserts a `<= 2.0s` threshold or evaluates transaction latency under normal concurrent store load.	**PARTIAL**	Single-user local SQLite execution completes well under 2 seconds, but no automated assertion or normal-load benchmark verifies the 2-second SLA under concurrent load.
NFR-002	System shall maintain 99% uptime during store operating hours. [Criteria: Uptime monitoring logs show >= 99% availability.]	None	No uptime monitoring logs, long-running availability test, or production reliability telemetry exists in the repository or test suite.	**NOT TESTED**	A working local application does not establish 99% availability; requires production deployment and continuous uptime monitoring logs.
NFR-003	Passwords shall be stored using secure hashing (e.g., bcrypt); RBAC enforced on all endpoints. [Criteria: No plaintext passwords in DB; restricted endpoints return 401/403 for unauthorized calls.]	IT_01, IT_02, IT_09, ST_03	Salted `bcrypt` hashing is verified in `tests/test_security.py` (6 tests). RBAC HTTP 403/302 checks are verified on `/users*`, `/suppliers*`, `/purchase-orders*`, `/reports/*`, and `/products/<id>/reorder-threshold`. However, product CRUD routes (`/products/create`, `/edit`, `/update`, `/delete`) in `main.py` do not enforce role decorators.	**PARTIAL**	Bcrypt password hashing and RBAC on users, suppliers, POs, reports, and reorder thresholds are verified, but RBAC is not enforced across all product CRUD routes.
NFR-004	UI shall allow a new staff member to complete a basic sale within 15 minutes of training. [Criteria: New user completes sale transaction unaided within 15 min.]	None	No human-participant usability study or timed staff onboarding trial has been conducted or recorded.	**NOT TESTED**	Requires empirical usability testing with new staff participants; cannot be verified by automated tests.
NFR-005	System shall ensure atomic transactions so stock and billing records never desynchronize. [Criteria: No partial updates occur even if a transaction fails mid-way.]	UT_02, IT_07, IT_10	User-reported manual pass (`UT_02`, `IT_07`, `IT_10`). Automated Pytest in `tests/test_sales.py` (`test_multi_item_sale_rolls_back_when_second_item_fails`, `test_sale_route_preserves_stock_on_failed_checkout`, `test_batch_restock_atomic_success_and_failure`), `tests/test_purchase_orders.py` (commit failure rollbacks), and `tests/test_products.py` (`test_failed_deletion_rolls_back_cleanly`).	**PASS**	Transactional atomicity and complete rollback on mid-transaction validation or commit failures are thoroughly verified.
NFR-006	System shall support up to 10,000 product records and 50 concurrent users without performance degradation. [Criteria: Load test with 50 concurrent users shows no crash or major slowdown.]	None	Automated tests run sequentially in a single process with small seeded datasets; `benchmarks/sales_performance.py` tests 1–100 products single-threaded. No 10,000-product or 50-concurrent-user load test has been executed.	**NOT TESTED**	Normal automated tests do not establish support for 10,000 product records or 50 concurrent users.
NFR-007	System shall be accessible via modern web browsers (Chrome, Firefox, Edge) without platform-specific installation. [Criteria: UI renders and functions correctly across all three browsers.]	None in 18-case PDF (README manual notes)	Automated Pytest uses Flask's non-browser `test_client()`. `README.md` documents user manual testing on Chrome 156.0.8078.79, Firefox 159.0, and Edge 156.0.4314.50 for 5 subset flows (login, inventory table, search/category filter, dynamic sale row, sale required fields), explicitly noting other flows were not verified.	**PARTIAL**	User-documented manual checks in `README.md` cover 5 basic flows across Chrome, Firefox, and Edge, but remaining modules and automated cross-browser tests were not executed.

SRS Req ID	Requirement Description & Acceptance Criteria	Related Test Case ID(s)	Verification Evidence	Status	Remarks
NFR-008	Codebase shall follow modular architecture (MVC) to allow independent module updates. [Criteria: Inventory, billing, and reporting modules can be updated independently without breaking others.]	UT_01, UT_02	Architecture inspection (`app/presentation/`, `app/business/`, `app/data/`) and service-level unit tests (`UT_01`, `UT_02`, `test_sales.py`, `test_purchase_orders.py`) confirm business logic separation. However, independent module modification/replacement without regression across shared models and `main.py` routes was not dynamically tested.	**PARTIAL**	Three-tier layered structure is verified by inspection and service tests, but independent update of inventory, billing, and reporting modules without cross-impact is not dynamically tested.
NFR-009	System shall perform automated daily database backups with a 24-hour recovery point objective. [Criteria: Backup file generated daily and successfully restorable within 24 hrs.]	IT_12	User-reported manual pass (`IT_12`). `tests/test_backup_database.py` (`test_create_backup_successfully`) verifies on-demand SQLite backup creation and `PRAGMA integrity_check == ('ok',)`. However, automated daily scheduling (cron/timer) and end-to-end restoration into a live application within a 24-hour RPO are not tested.	**PARTIAL**	On-demand backup generation and SQLite integrity check pass (`IT_12`), but automated daily scheduling and full database restoration within 24 hours are not verified.
NFR-010	System shall retain audit logs for a minimum of 6 months. [Criteria: Logs older than current date but within 6 months remain queryable.]	IT_06	`IT_06` verifies `AuditLog` creation. `tests/test_products.py` (`test_audit_logs_within_six_months_are_retained_and_queryable`, `test_audit_log_table_has_no_auto_purge_on_subsequent_actions`) verifies synthetic backdated entries (30, 90, 180 days) remain queryable in in-memory SQLite. Long-term persistent storage retention over 6 months is not verified.	**PARTIAL**	Audit log creation (`IT_06`), absence of auto-purge logic, and synthetic 180-day queryability in SQLite are verified, but 6-month operational retention in production storage is not tested.
NFR-011	System shall encrypt sensitive data (e.g., user credentials) in transit using HTTPS/TLS. [Criteria: All API/browser communication occurs over HTTPS; no plaintext transmission detected.]	None	Application runs on plain HTTP (`http://127.0.0.1:5000/`) in `run.py` and in-process WSGI `test_client()` in Pytest. No HTTPS/TLS configuration, HTTP-to-HTTPS redirect, or transport encryption test exists.	**NOT TESTED**	Password hashing (`test_security.py`) protects credentials at rest (`NFR-003`), not in transit. HTTPS/TLS encryption in transit is not configured or tested.
NFR-012	System shall provide clear, consistent error messages for all failed operations. [Criteria: Every failure case (invalid input, network error, etc.) shows a specific, user-understandable message.]	UT_02, IT_02, IT_04, IT_05	User-reported manual pass (`UT_02`, `IT_02`, `IT_04`, `IT_05`) and Pytest modules verify clear validation messages for invalid inputs, unauthorized roles, overselling, over-delivery, and simulated commit errors. However, network error UI handling and uniform generic error sanitization across all endpoints (`SEC-012`) are not fully tested.	**PARTIAL**	Input validation and business-rule error messages are verified across all modules, but network failure handling and complete `SEC-012` database error sanitization are only partially verified.

7. Validation Results Summary

Because the 24 SRS requirements and the 18 formal test cases measure different verification dimensions, their results are summarized in separate tables below. Table 4 provides the status distribution across all 24 SRS requirements (`FR-001`–`FR-012` and `NFR-001`–`NFR-012`), and Table 5 provides the status distribution across the 18 test cases in `Documents/test_plan.pdf`.

7.1 SRS Requirements Validation Summary (24 Requirements)

Status	Functional (FR-001–FR-012)	Non-Functional (NFR-001–NFR-012)	Requirement Count	Requirement IDs
PASS	12	1	**13**	`FR-001` through `FR-012`, `NFR-005`
PARTIAL	0	7	**7**	`NFR-001`, `NFR-003`, `NFR-007`, `NFR-008`, `NFR-009`, `NFR-010`, `NFR-012`
FAIL	0	0	**0**	None
NOT TESTED	0	4	**4**	`NFR-002`, `NFR-004`, `NFR-006`, `NFR-011`
Total	**12**	**12**	**24**	**All 24 SRS Requirements Accounted For**

7.2 18-Case Test Document Summary (18 Test Cases)

Status	Unit Testing (UT)	Integration Testing (IT)	System Testing (ST)	Test Case Count	Test Case IDs
PASS	2	12	4	**18**	`UT_01`–`UT_02`, `IT_01`–`IT_12`, `ST_01`–`ST_04`
PARTIAL	0	0	0	**0**	None (Note: `IT_01` dashboard redirect verified manually; automated test checks session)
FAIL	0	0	0	**0**	None
NOT TESTED	0	0	0	**0**	None
Total	**2**	**12**	**4**	**18**	**All 18 Test Cases Accounted For**

8. Defects, Limitations, and Untested Requirements

1. Observed Test Failures: Zero (`0`) test failures or errors occurred across the 373 executed Pytest tests, and zero (`0`) test failures were reported across the 18 test cases in `Documents/test_plan.pdf`.

2. Partially Verified SRS Requirements (7 Requirements):

- `NFR-001` **(Sale & Inventory Update ≤ 2 Seconds):** While single-user sales execute in milliseconds in Pytest and in `benchmarks/sales_performance.py`, no automated test enforces a `<= 2.0s` assertion or measures latency under realistic concurrent store load.
- `NFR-003` **(Password Hashing & Endpoint RBAC):** Salted `bcrypt` storage (`tests/test_security.py`) and role decorators on `/users*`, `/suppliers*`, `/purchase-orders*`, `/reports/*`, and `/products/<id>/reorder-threshold` are verified. However, product CRUD routes (`/products/create`, `/edit`, `/update`, `/delete`) in `app/presentation/routes/main.py` lack server-side role decorators (`@admin_required`), and additional security controls in `Documents/security_requirements_specifications.pdf` (`SEC-002` password complexity, `SEC-004` 5-attempt lockout, `SEC-005` 30-minute idle timeout, `SEC-011` single-use reset token invalidation) are not implemented or tested.
- `NFR-007` **(Cross-Browser Compatibility):** `README.md` records manual testing on Chrome 156, Firefox 159, and Edge 156 for 5 basic flows, but purchase orders, suppliers, returns, reports, and user administration were not verified across browsers, and Pytest uses non-browser `test_client()`.
- `NFR-008` **(Modular Architecture):** Three-tier package separation (`presentation`, `business`, `data`) is confirmed by code inspection and service unit tests (`UT_01`, `UT_02`), but dynamic module-independence testing was not performed and `main.py` combines product, sales, return, and report routes in a single blueprint.
- `NFR-009` **(Automated Daily Backups & 24h Recovery):** `IT_12` and `tests/test_backup_database.py` verify on-demand backup creation and `PRAGMA integrity_check`, but automated daily cron/scheduler execution, encrypted storage (`SEC-014`),

and full database restoration within 24 hours are not tested.

- `NFR-010` **(Six-Month Audit Log Retention):** `IT_06` verifies log creation and `tests/test_products.py` verifies that synthetic backdated records (30, 90, 180 days) remain queryable in in-memory SQLite, but actual 6-month retention in persistent production storage is not verified.
- `NFR-012` **(Clear & Consistent Error Messages):** Input and domain validation errors are verified across all routes, but client-side network error handling and uniform generic database error masking (`SEC-012`) are not comprehensively verified.

3. Untested SRS Requirements (4 Requirements):

- `NFR-002` **(99% Uptime During Operating Hours):** Not tested; requires production deployment and continuous availability monitoring logs.
- `NFR-004` **(New Staff Usability Within 15 Minutes of Training):** Not tested; requires an empirical human-factors usability study with new staff members.
- `NFR-006` **(10,000 Products and 50 Concurrent Users):** Not tested; no high-volume dataset (10,000 products) or concurrent multi-user load test (50 simultaneous users) has been executed.
- `NFR-011` **(HTTPS/TLS Encryption in Transit):** Not tested; the application runs over plain HTTP in local development and in-process WSGI `test_client()` during testing, with no TLS termination or HTTPS redirect (`SEC-008`).

4. Validation Environment Limitations: (a) All 373 automated tests execute against ephemeral SQLite databases (`sqlite:///:memory:` and `tmp_path`) rather than a networked MySQL server (`PyMySQL`); (b) Flask's `test_client()` bypasses real TCP/TLS transport and browser DOM/JavaScript execution; (c) outbound password-reset email dispatch is verified via `unittest.mock.patch` rather than a live SMTP server; and (d) in the Revenue Report (`FR-009`), gross sales revenue is aggregated without subtracting `ProductReturn` refunds (which the UI explicitly discloses via a limitation banner).

9. Overall Conclusion

The combined validation evidence—comprising **373 passing automated Pytest tests** (0 failed, 0 errors, 0 skipped in 105.34s) and **18 passing test cases** in `Documents/test_plan.pdf` (supported by user-reported manual execution and automated test mappings)—establishes that the **core functional baseline** of the Electronic Store Inventory Management System is thoroughly verified. Specifically:

- **Verified Functional & Transactional Requirements (13 PASS):** All 12 Functional Requirements (`FR-001` through `FR-012`) and `NFR-005` (Atomic Transactions) are verified by concrete automated test evidence (and, for the cases represented in `Documents/test_plan.pdf`, user-reported manual verification). Note that `FR-006` (Supplier CRUD & product linking), `FR-008` (Product search & filtering), and `FR-012` (Password reset) are validated through dedicated automated Pytest modules (`test_suppliers.py`, `test_products.py`, `test_auth.py`) even though they are not separately represented as standalone cases in the 18-case test document.
- **Partially Verified Non-Functional Requirements (7 PARTIAL):** `NFR-001` (2-second sale latency), `NFR-003` (bcrypt hashing & universal endpoint RBAC), `NFR-007` (cross-browser compatibility), `NFR-008` (modular independence), `NFR-009` (daily backups & 24-hour recovery), `NFR-010` (6-month audit retention), and `NFR-012` (consistent error handling) are partially verified: their core mechanisms work in local testing, but full production acceptance criteria (such as RBAC decorators on product CRUD routes, automated daily backup scheduling and restoration testing, and full cross-browser coverage) remain incomplete.
- **Untested Non-Functional Requirements (4 NOT TESTED):** `NFR-002` (99% uptime), `NFR-004` (15-minute new staff usability), `NFR-006` (10,000 products and 50 concurrent users), and `NFR-011` (HTTPS/TLS encryption in transit) have no execution evidence.
- **Final Assessment on SRS Validation Completeness:** While the available evidence is sufficient to confirm that all 12 functional requirements (`FR-001`–`FR-012`) and transactional atomicity (`NFR-005`) meet specification, it is **not sufficient to claim complete validation of all 24 SRS requirements**. Achieving full SRS sign-off requires enforcing role checks on product CRUD routes, executing a 50-user / 10,000-product load test, performing an end-to-end backup restoration test, conducting a staff usability trial, deploying behind HTTPS/TLS, and collecting production uptime logs.